

Pac-man Assembly Project 보고서

남궁명수

I. 서론.....	2
II. 프로젝트 목표.....	2
III. 전체 개발 전략.....	3
[1] 개발 전략 개요.....	3
[2] NASM Only 텍스트 Pacman.....	3
Chapter 1. 프로젝트 개발 환경.....	5
[1] 개발 시스템 개요.....	5
[2] 하드웨어 환경.....	5
[3] 운영체제 환경.....	5
[4] 개발 도구 구성.....	5
Chapter 2. NASM Only 텍스트 Pacman.....	6
[1] 개발 환경 세팅.....	6
[2] Text Pacman 설계.....	9
[3] 파일 구조.....	10
[4] 터미널 기반 인터랙티브 화면 제어 구조.....	10
Chapter 3. 코드.....	13
[1] game_loop.asm.....	13
[2] input.asm.....	20
[3] terminal.asm.....	20
[4] map.asm.....	23
[5] player.asm.....	24
[6] ghost.asm.....	29
Chapter 4. 결론 및 한계.....	37
결론.....	37
한계.....	37
부록. 기술 설명.....	38
[1] 리눅스 터미널 제어와 Raw Mode 구현 분석.....	38
[2] ANSI Escape Sequence 기반 커서 이동 루틴의 구조 및 자릿수 처리 원리 분석.....	43
[3] 정수 → 10 진수 문자열 변환.....	51
[4] 프레임 제한과 나노초 단위 시간.....	54

I. 서론

본 프로젝트는 x86 아키텍처 기반의 저사양 데스크탑 환경에서 터미널 기반 Text Pacman 게임을 구현함으로써, 어셈블리 언어에 대한 실질적이고 구조적인 이해를 심화하기 위해 기획되었다.

현대 소프트웨어 개발은 대부분 고수준 언어와 고성능 하드웨어 환경을 전제로 이루어지고 있다. 그 결과, 프로그램이 실제 하드웨어 위에서 어떻게 동작하는지에 대한 근본적인 이해는 상대적으로 간과되기 쉽다. 본인 역시 대학 과정에서 프로그래밍을 학습하였으나, 어셈블리 언어를 접하기 전까지는 저수준 시스템 동작 원리에 대해 깊이 있는 이해가 부족하였다.

이에 따라 본 프로젝트는 과거 개발자들이 제한된 하드웨어 환경 속에서 어셈블리 언어만으로 프로그램을 설계하고 구현해왔던 방식을 직접 체험하기 위한 목적에서 출발하였다. 비록 미숙한 단계이지만, 로우레벨 환경에서의 개발 환경을 통해 당시 개발자들의 설계 방식과 문제 해결 접근법을 추적하고 이해하고자 한다.

본 프로젝트는 다음과 같은 저수준 개념에 대한 이해 향상을 목표로 한다.

- 메모리 구조 및 데이터 배치 방식
- 레지스터 활용 전략
- 함수 호출 규약
- C 언어와 어셈블리 간 인터페이스 구조
- 바이너리 레벨에서의 실행 흐름

이를 위해 저사양 x86 시스템 환경을 기반으로 NASM 어셈블리 언어를 단순한 문법 학습 도구가 아닌 실제 실행 가능한 프로그램의 핵심 구성 요소로 활용한다.

II. 프로젝트 목표

프로젝트의 목표는 *x86-64 Assembly Language Programming with Ubuntu* 를 기반으로 학습한 내용을 NASM 기반 x86 어셈블리 언어를 실제 프로젝트에 적용하여, 실전 수준의 코드 작성 경험을 축적하는 것이다.

또한 로우레벨 프로그래밍의 역량 강화와 시스템적 사고 확장을 궁극적인 목표로 한다.

III. 전체 개발 전략

[1] 개발 전략 개요

이 프로젝트는 단계적 난이도 상승 구조를 기반으로 최종적으로 구조화된 게임 엔진 형태의 구현을 목표로 한다.

개발은 단순 기능 구현에서 시작하여 점진적으로 시스템 구조를 확장하는 방식으로 진행되며, 다음과 같이 구성된다.

- 순수 NASM 기반 텍스트 Pacman 구현

1 단계는 로직 중심 학습 단계이며,

2 단계는 구조 확장 및 엔진 설계 단계이다.

[2] NASM Only 텍스트 Pacman

[2-1] 목적

어셈블리 언어 기반 저수준 프로그래밍 능력을 강화하는 것이다. 주요 목표는 다음과 같다.

- x86-64 어셈블리 문법 숙달
- syscall 기반 입출력 이해
- 메모리 구조 및 배열 처리 능력 향상
- 게임 로직을 그래픽 없이 순수 알고리즘으로 구현

이 단계에서는 C 언어 및 SDL2 를 사용하지 않는다.

[2-2] 개발 환경

- Ubuntu 22.04 x86_64
- NASM
- ld(binutils)
- Linux syscall 인터페이스

[2-3] 시스템 구조

텍스트 Pacman 은 단일 실행 파일 기반의 모듈형 구조로 구성된다.

```
_start
├── src
│   ├── game_loop.asm
│   ├── ghost.asm
│   ├── input.asm
│   ├── map.asm
│   ├── player.asm
│   └── terminal.asm
```

[2-4] 구현 목표

Step 1: 맵 출력

- 2 차원 맵을 1 차원 배열로 정의한다.
- `sys_write` 를 사용하여 전체 맵을 터미널에 출력한다.

Step 2: 입력 처리

- `sys_read` 를 이용하여 1 바이트 입력을 수신한다.
- W, A, S, D 키를 기반으로 이동 방향을 제어한다.

Step 3: 위치 갱신

- 플레이어 좌표를 변수로 관리한다.
- 입력된 방향에 따라 좌표 값을 갱신한다.

Step 4: 충돌 검사

- 인덱스 계산: $\text{index} = y * \text{width} + x$
- 해당 위치가 벽('#')인 경우 이동을 취소한다.

Step 5: 점수 시스템

- 아이템(.)을 획득하면 해당 문자를 제거한다.
- 점수를 증가시키는 로직을 추가한다.

Step 6: 고스트

- 단순 랜덤으로 이동한다.
- 플레이어와의 충돌 로직을 추가한다.

Chapter 1. 프로젝트 개발 환경

[1] 개발 시스템 개요

본 프로젝트는 2009 년 출시된 x86 기반 데스크탑 PC 환경에서 진행된다.
해당 환경은 현대적인 고성능 시스템 대비 제한된 연산 성능과 메모리 자원을 가지고 있어,
어셈블리 기반 최적화 및 시스템 이해를 심화하기에 적합하다고 판단했다.

[2] 하드웨어 환경

구분	내용
모델명	삼성 매직스테이션 DM-C200-PAD1
제조사	삼성전자
출시 시기	2009년
CPU	Intel Pentium E5400
CPU 아키텍처	x86 (64-bit 지원)
동작 속도	2.7 GHz
메모리	2GB DDR3
저장 장치	250GB SATA HDD
그래픽	Intel GMA X4500 (내장 그래픽)

[3] 운영체제 환경

항목	내용
운영체제	Ubuntu 22.04.5 LTS
배포판	Ubuntu
릴리즈	22.04
코드네임	jammy
시스템 유형	x86_64 Linux

[4] 개발 도구 구성

도구	역할	비고
nasm	x86 어셈블러	저수준 코드 작성
gcc	C 컴파일러	SDL 기반 게임 구조 구현
ld (binutils)	링커	오브젝트 파일 연결
make	빌드 자동화	프로젝트 관리
pkg-config	라이브러리 설정	SDL2 컴파일 옵션 관리

Chapter 2. NASM Only 텍스트 Pacman

[1] 개발 환경 세팅

- 순수 NASM 으로 ELF64 실행 파일 생성
- Linux syscall 기반 실행
- ld 로 직접 링크
- Makefile 없이도 빌드 가능 상태 확보

[1-1] 필수 패키지 설치

```
sudo apt update  
sudo apt install nasm binutils make gdb
```

도구	역할
nasm	어셈블러
ld (binutils)	링커
make	빌드 자동화
gdb	디버깅

[1-2] 설치 확인

```
nasm -v  
ld -v  
gdb --version
```

[1-3] 작업 디렉토리 생성

```
mkdir ~/pacman_asm  
cd ~/pacman_asm
```

[1-4] 최소 실행 테스트

main.asm 생성

```
nano main.asm
```

아래 코드 입력:

```
section .data
msg db "Hello Pacman", 10
len equ $ - msg

section .text
global _start

_start:
    mov rax, 1          ; sys_write
    mov rdi, 1          ; stdout
    mov rsi, msg        ; address
    mov rdx, len        ; length
    syscall

    mov rax, 60         ; sys_exit
    xor rdi, rdi        ; status 0
    syscall
```

[1-5] 빌드

```
nasm -f elf64 -g -F dwarf main.asm -o main.o
```

-f : outputfile format(출력 파일 형식)을 지정하는 옵션

-g : 디버깅 심볼 포함

-F dwarf: gdb 에서 읽을 수 있는 디버깅 정보 형식 지정

[1-6] 링크

```
ld main.o -o pacman
```

[1-7] 실행

```
./pacman
```

[1-8] 디버깅 테스트

```
gdb ./pacman
```

gdb 프롬프트가 뜬다.

[1-9] 기본 디버깅 명령

[1] 브레이크 포인트 설정

```
break _start
```

_start 라벨에서 멈춘다.

[2] 실행 시작

run

프로그램이 _start 에서 멈춘다.

[3] 한 줄씩 실행

si

si : CPU 명령어 단위로 진행

nexti(ni)도 유사하게 사용 가능

[4] 레지스터 확인

info registers

rax, rdi, rsi, rdx 등 시스템 콜에 사용되는 레지스터 상태 확인 가능

p/x \$rax

p \$rsi

rax 값 16 진수로 보기

rsi 값 보기

[5] 메모리 확인

x/s map

x/16xb clear_screen

x: examine(확인)의 약자

/s: 문자열

/xb: 16 바이트 단위 16 진수

[6] 프로그램 계속 실행

c

멈춘 곳 이후로 계속 실행

[2] Text Pacman 설계

[2-1] 기능 범위 정의

포함 기능

- 고정 맵 사용
- 플레이어 이동(W, A, S, D)
- 벽 충돌 처리
- ANSI escape 코드 기반 화면 전체 재출력
- 'q' 입력 시 종료
- 점수 시스템
- 점수 먹기 기능
- 실시간 입력(raw mode)
- 유령

[2-2] 전체 구조 설계 요약

[2-2-1] 프로그램 흐름

초기화

↓

무한 루프 (game_loop)

- └ 입력
- └ 이전 상태 삭제
- └ 플레이어 이동
- └ 고스트 이동
- └ 충돌 검사
- └ 승리 검사
- └ 다시 그리기
- └ 딜레이

↓

종료 (q / 충돌 / 승리)

- 종료 조건: 입력이 'q'일 경우 exit syscall 호출

[3] 파일 구조

```
text_pacman_v0.1/
├── src/                # 모든 어셈블리 코드
│   ├── map.asm         # 맵 정의 + 출력
│   ├── player.asm      # 플레이어 좌표 정의
│   ├── terminal.asm     # Raw Mode 사용
│   ├── input.asm        # 입력 처리
│   ├── ghost.asm        # 고스트 좌표 정의 + 이동 + 충돌
│   └── game_loop.asm    # 게임 루프
│
├── build/              # 빌드 산출물
│   └── *.o              # object 파일
│
└── Makefile             # 빌드 자동화
```

[4] 터미널 기반 인터랙티브 화면 제어 구조

[4-1] 터미널 화면 제어 방식의 단계적 발전 과정

터미널을 단순 출력 창이 아니라 좌표 기반 화면 시스템으로 사용한다.

이를 위해 ANSI Escape Sequence를 활용하여 화면을 제어하였다.

해당 과정은 다음과 같은 단계로 발전하였다.

[4-1-1] 1 단계: 전체 화면 삭제 후 전체 재출력 방식

사용한 명령어

```
ESC[2J ; 화면 전체 삭제
ESC[H ; 커서를 (1, 1)로 이동
```

예시:

```
printf("\x1b[2J");
printf("\x1b[H");
```

동작 원리

1. 전체 화면을 지운다.
2. 커서를 홈 위치(1 행 1 열)로 이동한다.
3. 화면 전체 내용을 다시 출력한다.

특징

- 구현이 단순하다.
- 매 프레임 모든 내용을 다시 출력한다.
- 화면 깜빡임(flicker)이 발생할 수 있다.
- CPU 및 출력 비용이 비교적 크다.

이 방식은 초기 텍스트 게임이나 단순 콘솔 프로그램에서 자주 사용된다.

해당 제어 시퀀스는 ECMA-48 표준에 정의되어 있으며, 대부분의 리눅스 터미널은 XTerm 규격을 따른다.

[4-1-2] 2 단계: 화면 삭제 없이 커서 이동만 활용 사용한 명령어

ESC[H ; 커서를 홈 위치로 이동

동작 원리

1. 화면 전체를 지우지 않는다.
2. 커서를 특정 위치로 이동시킨다.
3. 필요한 내용만 다시 출력한다.

특징

- 전체 삭제를 하지 않기 때문에 깜빡임 감소
- 출력 비용 감소
- 이전 출력 내용이 남아있을 수 있으므로 관리 필요

[4-1-3] 3 단계: 좌표 기반 직접 커서 제어 방식 사용한 명령

ESC[y;xH

예시:

ESC[10;20H

10 행 20 열로 커서 이동

동작 원리

1. 커서를 원하는 좌표로 직접 이동한다.
2. 변경이 필요한 위치에만 문자를 출력한다.
3. 전체 화면을 다시 그리지 않는다.

특징

- 필요한 부분만 갱신(Partial Redraw)
- 깜빡임 거의 없음
- 효율적인 렌더링
- 콘솔 게임 및 텍스트 UI 구현의 핵심 기술

이 방식은 터미널을 2 차원 좌표 기반 디스플레이처럼 사용하는 것이다.

Chapter 3. 코드

[1] game_loop.asm

[1-1] 구조

```
초기화
↓
무한 루프 (game_loop)
├ 입력
├ 이전 상태 삭제
├ 플레이어 이동
├ 고스트 이동
├ 충돌 검사
├ 승리 검사
├ 다시 그리기
└ 딜레이
↓
종료 (q / 충돌 / 승리)
```

[1-1-1] 입력 → 삭제 → 이동 그리기 순서의 이유

게임 루프는 기본적으로 “매 프레임 화면을 새로 그리는 방식”으로 동작한다.

즉, 이전 화면을 유지하면서 조금씩 수정하는 것이 아니라, 매 순간 전체 화면을 다시 구성하는 구조이다.

먼저 입력 처리 단계에서는 사용자의 키 입력을 받아 현재 프레임에서 어떤 행동을 할지 결정한다. 예를 들어, W 키가 눌렸다면 위로 이동, A 키가 눌렸다면 왼쪽으로 이동하는 식으로 “이번 프레임의 이동 방향”을 계산한다.

그 다음 이전 상태를 삭제하는 단계가 필요하다.

이는 화면이 누적되는 방식이 아니라 매 프레임 새로 그려지는 방식이기 때문이다.

만약 이전 프레임의 그림을 지우지 않으면 캐릭터가 이동할 때마다 이전 위치가 남아서 잔상이 생기고, 결국 화면이 깨지게 된다.

이후 이동 단계에서 입력값을 기반으로 캐릭터의 새로운 위치를 업데이트 한다.

마지막으로 그리기 단계에서 업데이트된 위치를 기준으로 화면을 다시 렌더링한다.

즉, 입력을 통해 상태를 결정하고, 이전 프레임을 초기화한 후, 새로운 상태를 계산하고 이를 다시 렌더링하는 구조로 동작한다.

[1-2] 주요 패턴

시스템 콜 패턴

```
mov rax, SYS_write    ; syscall 번호
mov rdi, STDOUT        ; arg1
mov rsi, buffer        ; arg3
mov rdx, length        ; arg3
```

화면 제어(ANSI escape)

ANSI escape

- 터미널 에뮬레이터에서 텍스트의 색상, 글꼴, 스타일, 커서 위치 등을 제공하기 위해 사용하는 일련의 특수 문자열 표준이다.

```
clear_screen db 27, '[', '2', 'J', 27, '[', 'H'
```

- ESC [2J : 화면 전체 클리어
- ESC [H : 커서 홈

```
hide_cursor db 27, '[', '?', '2', '5', 'l'
```

- ESC [?25l
 - ?25: 커서 표시/숨김 제어
 - l : 비활성화

```
show_cursor db 27, '[', '?', '2', '5', 'h'
```

- ESC [?25h
 - [?25: 커서 표시/숨김 제어
 - h : 활성화

프레임 딜레이

```
frame_delay:
    dq 0
    dq 1000000000
```

- 부록: [4] 프레임 제한과 나노초 단위 시간

[1-3] main 함수

main:

 call enable_raw_mode

– 터미널을 raw mode 로 바꿈

커서 숨기기

mov rax, SYS_write

mov rdi, STDOUT

mov rsi, hide_cursor

mov rdx, cursor_ctl_len

syscall

화면 클리어

mov rsi, clear_screen

초기 게임 세팅

call init_dot_count

call draw_map

call draw_player

call draw_ghost

call draw_score

[1-4] 게임 루프

```
game_loop:
```

입력 읽기

```
call read_input  
mov al, byte [input_char]  
or al, 0x20
```

- 함수의 반환값이 eax
- OR 0x20 → 대문자 → 소문자 변환
- 'A' (0x41) → 'a' (0x61)
- 32bit: EAX
- EAX 의 하위 16 비트: AX
- AX 의 상위 8 비트: AH
- AX 의 하위 8 비트: AL

종료

```
cmp al, 'q'  
je exit_program
```

- q 누르면 종료
- je(Jump if Equal) : 두 값이 같으면 점프

이전 상태 지우기

```
call erase_player  
call erase_ghost
```

플레이어 이동

```
call update_player
```

```
call check_ghost_collision  
test rax, rax  
jnz game_over
```

- check_ghost_collision:
 - rax = 0: 충돌 없음
 - rax = 1: 충돌 있음
- rax AND rax
- jnz(Jump if Not Zero): rax != 0 이면 종료


```
call check_ghost_cross
test rax, rax
    jnz game_over
```

위와 동일한 방식

고스트 이동

```
call move_ghost

    call check_ghost_collision
    test rax, rax
    jnz game_over

    call check_ghost_cross
    test rax, rax
    jnz game_over
```

승리 조건

```
call check_win
test rax, rax
jnz win_program
```

- dot 다 먹으면 승리

다시 그리기

```
call draw_player
call draw_ghost
call draw_score
```

프레임 제한

```
mov rax, SYS_nanosleep
mov rdi, frame_delay
xor rsi, rsi
syscall
```

- 부록: [4] 프레임 제한과 나노초 단위 시간

루프 반복

```
jmp game_loop
```

[1-5] 게임 오버

터미널 복구

```
game_over:
    call disable_raw_mode
```

메시지 출력

```
mov rax, SYS_write
mov rdi, STDOUT
mov rsi, game_over_msg
mov rdx, game_over_len
syscall
```

커서 복구

```
mov rax, SYS_write
mov rdi, STDOUT
mov rsi, show_cursor
mov rdx, cursor_ctl_len
syscall
```

종료

```
mov rax, SYS_exit
xor rdi, rdi
syscall
```

- rdi 를 0 으로 만들기
- rdi = 종료 코드(exit code)

[1-6] 승리 처리

터미널 복구

```
win_program:
    call disable_raw_mode
```

화면 아래에 WIN 출력

```
mov rax, SYS_write
mov rdi, STDOUT
mov rsi, win_msg
mov rdx, win_msg_len
syscall
```

커서 다시 보이기

```
mov rax, SYS_write
mov rdi, STDOUT
mov rsi, show_cursor
mov rdx, cursor_ctl_len
syscall
```

종료

```
mov rax, SYS_exit
xor rdi, rdi
syscall
```

[1-7] 정상 종료

```
exit_program:
    call disable_raw_mode

mov rax, SYS_write
mov rdi, STDOUT
mov rsi, show_cursor
mov rdx, cursor_ctl_len
syscall

mov rax, SYS_exit
xor rdi, rdi
syscall
```

[2] input.asm

```
mov rax, SYS_read
mov rdi, STDIN      ; standard input
mov rsi, input_char
mov rdx, 1
```

- 사용자로부터 입력받은 값은 초기화되지 않은 메모리 영역인 .bss 에 선언된 input_char 변수에 저장된다.
- 또한, 해당 변수는 global 로 선언되어 다른 파일에서도 접근 및 사용이 가능하다.

[3] terminal.asm

[3-1] 구조

```
enable_raw_mode  → 터미널 "게임 모드"
disable_raw_mode → 원래 상태 복구
```

Canonical mode(기본)

- 엔터를 눌러야 입력 전달
- echo 켜져 있음(입력 화면에 보임)
 - echo = 내가 누른 키를 화면에 보여주는 기능

Raw mode(게임용)

- 키 누르는 순간 즉시 입력
- echo 꺼짐
- 버퍼링 없음

.bss 영역

```
section .bss
old_termios resb 60
newtermios resb 60
```

- libc 구조체 termios 저장 공간
 - libc = C 표준 라이브러리(예: printf)
- resb: 60byte 의 공간만 예약, 초기화 x

[3-2] enable_raw_mode

현재 터미널 상태 가져오기

```
mov rdi, 0
mov rsi, old_termios
call tcgetattr
```

c 로 보면

```
tcgetattr(0, &old_termios);
```

- 현재 설정 읽기

구조체 복사

```
mov rcx, 60
mov rsi, old_termios
mov rdi, new_termios
```

- 60 바이트 통째로 복사

루프

```
.copy:
    mov al, [rsi]
    mov [rdi], al
    inc rsi
    inc rdi
    loop .copy
```

- rcx = rcx - 1

- if rcx != 0 → jump

- mov al, [rsi]

- [rsi] = 이 주소에 있는 실제 데이터

- rsi 가 가리키는 메모리 1 바이트를 al 에 넣어라

- mov [rdi], al

- al 에 들어 있는 내용을 rdi 에 저장

raw mode 핵심 비트 수정

```
mov eax, [new_termios + 12]
and eax, ~0x000A
mov [new_termios + 12], eax
```

설정 적용

```
mov rdi, 0
mov rsi, 0
mov rdx, new_termios
call tcsetattr
```

- tcsetattr: 새 설정 적용

[3-3] disable_raw_mode

원래 터미널 복구

```
mov rdi, 0
mov rsi, 0
mov rdx, old_termios
call tcsetattr
```

[4] map.asm

[4-1] 전체 구조

map_data → 맵 문자열 데이터 draw_map → 화면에 출력

- 게임 배경을 단순 문자열로 렌더링

[4-2] draw_map 함수

draw_map: mov rax, SYS_write mov rdi, STDOUT mov rsi, map_data mov rdx, map_size syscall ret
--

[5] player.asm

[5-1] 구조

1. 플레이어 좌표 관리 (x, y)
2. 화면 렌더링 (draw / erase)
3. 입력 기반 이동 (WASD)
4. 맵 충돌 처리
5. 점수 시스템
6. 승리 조건

[5-2] 데이터 구조

플레이어 상태

```
player_x db 2  
player_y db 2
```

- byte(0~255)
- 맵 좌표용(0~9)

점수 시스템

```
score dq 0  
total_dots dq 0
```

- score: 먹은점
- total_dots: 남은점

[5-3] move_cursor

이 함수는 (y, x)를 받아서 ESC[y;xH 문자열을 만들어 터미널 커서를 이동시키는 코드이다.

[5-4] draw_player

커서 이동

```
movzx rdi, byte [player_y]  
movzx rsi, byte [player_x]  
call move_cursor
```

'P' 출력

```
mov rax, SYS_write  
    mov rdi, STDOUT  
    lea rsi, [player_char]  
    mov rdx, 1  
    syscall
```


[5-5] erase_player

위에서 설명한 잔상 제거 방식은 erase → update → draw 로 동작한다.

```
movzx rdi, byte [player_y]
movzx rsi, byte [player_x]
call move_cursor

mov rax, SYS_write
mov rdi, STDOUT
lea rsi, [space_char]
mov rdx, 1
syscall
```

- lea(Load Effective Address): 메모리 값을 읽는게 아닌, 주소 자체를 계산해서 레지스터에 넣는 명령

[5-6] calc_index

$\text{index} = y * (\text{width} + 1) + x$

```
mov rcx, [map_width]
inc rcx
imul rax, rcx
add rax, rbx
```

- 줄 바꿈 문자로 인해 row stride 가 증가했기 때문에 +1 을 한다.

[5-7] update_player

현재 좌표 로드

```
movzx rax, byte [player_y]
movzx rbx, byte [player_x]
```

- rax = y
- rbx = x

입력 처리

```
mov cl, [input_char]
or cl, 0x20
```

- 대문자라면 소문자로 변환

방향 분기

```
cmp cl, 'w'
je try_up
```

이동 로직(Up)

```
cmp rax, 0
je done
dec rax
```

- 맵 밖 방지

down

```
mov rcx, [map_height]
dec rcx
cmp rax, rcx
```

- 하단 경계 체크

[5-8] check_wall

현재 상태 저장

```
push rax
push rbx
```

- 이동 후보 보존

맵 접근

```
call calc_index
mov dl, [map_data + rax]
```

- (x, y) → 1D index 변환 → tile 읽기

벽 체크

```
cmp dl, '#'
je cancel_move
```

- 벽이면 이동 취소

점처리

```
cmp dl, '.'
jne .no_dot

mov byte [map_data+rax], ''
inc qword [score]
dec qword [total_dots]
```

- 점이면: 제거/점수+1/남은 점 -1

이동 확정

```
.no_dot:
    pop rbx
    pop rax

    mov [player_y], al
    mov [player_x], bl
    ret
```

- 플레이어 위치 갱신

이동 취소

```
cancel_move:
    pop rbx
    pop rax
    ret
```

[5-9] init_dot_count(맵 점 개수 세기)

게임 시작 시 ‘.’ 개수 계산

```
init_dot_count:
    mov rcx, map_size
    lea rsi, [map_data]
    xor rax, rax
```

- rcx = 전체길이
- rsi = 맵 포인터
- rax = count

루프

```
.loop:
    cmp rcx, 0
    je .done

    mov dl, [rsi]
```

점 발견

```
cmp dl, '.'
jne .next

inc rax
```

다음 문자

```
.next:
    inc rsi
    dec rcx
    jmp .loop
```

종료

```
.done:
    mov [total_dots], rax
    ret
```

[5-10] check_win

점이 0 이면 승리

```
check_win:
    mov rax, [total_dots]
    cmp rax, 0
    jne .not_win
```

승리

```
mov rax, 1
ret
```

아직 진행 중

```
.not_win:
    xor rax, rax
    ret
```

[6] ghost.asm

해당 파트는 구현 과정에서 복잡한 오류를 해결하기 위해 AI 도구(ChatGPT)의 도움을 참고하여 개발을 진행한 영역이다.

초기 구현에서는 고스트 이동 과정에서 맵 데이터가 손상되는 문제와 플레이어와의 충돌이 정상적으로 처리되지 않는 오류가 지속적으로 발생하였다.

이에 따라 문제 원인을 분석하는 과정에서 다양한 접근 방식을 검토하였으며, AI의 제안을 참고하여 상태 저장 및 복원 구조와 충돌 판정 로직을 개선하였다. 이후 추가적인 검증과 수정을 거쳐 최종적으로 안정적인 동작을 구현하였다.

[6-1] 구조

- | | |
|-----------------|----------------|
| 1. draw_ghost | → 현재 위치 표시 |
| 2. erase_ghost | → 이전 위치 복원 |
| 3. move_ghost | → 랜덤 이동 + 벽 체크 |
| 4. collision | → 플레이어 충돌 |
| 5. cross check | → 스쳐 지나감 체크 |
| 6. is_wall | → 벽 판정 |
| 7. get_map_char | → 맵 문자 읽기 |

[6-2] draw_ghost

현재 위치의 원래 맵 문자 저장

<pre>mov al, [ghost_y] mov bl, [ghost_x] call get_map_char mov [ghost_prev_char], al</pre>
--

- 저장 이유: ghost가 지나가면 맵이 깨지지 않게 복원하기 위함

출력

<pre>movzx rdi, byte [ghost_y] movzx rsi, byte [ghost_x] call move_cursor mov rax, SYS_write mov rdi, STDOUT mov rsi, ghost_char mov rdx, 1 syscall</pre>
--

[6-3] erase_ghost

이전 위치를 원래 문자로 되돌림

```
erase_ghost:
movzx rdi, byte [ghost_prev_y]
movzx rsi, byte [ghost_prev_x]
call move_cursor

mov rax, SYS_write
mov rdi, STDOUT
lea rsi, [ghost_prev_char]
mov rdx, 1
syscall
ret
```

[6-4] move_ghost

고스트가 랜덤 방향으로 이동하되, 벽이면 이동 안함

이전 위치 저장

```
mov al, [ghost_x]
mov [ghost_prev_x], al

mov al, [ghost_y]
mov [ghost_prev_y], al
```

- 현재 위치를 backup
- 나중에 되돌리거나 렌더링용

이전 위치 맵 문자 저장

```
mov al, [ghost_prev_y]
mov bl, [ghost_prev_x]
call get_map_char
mov [ghost_prev_char], al
```

랜덤 방향 생성

```
rdtsc  
shr eax, 3  
and eax, 3
```

- rdtsc: CPU clock 기반 값
 - eax = 매우 큰 증가값
- shr eax, 3: 랜덤값을 줄여서 안정화
- and eax, 3: 0~3 범위로 제한
- 따라서, up(0), down(1), left(2), right(3)

방향 분기

```
cmp eax, 0  
je ghost_up  
  
cmp eax, 1  
je ghost_down  
  
cmp eax, 2  
je ghost_left  
  
jmp ghost_right
```

[6-5] 이동(위, 아래, 좌, 우)

공통 패턴

1. 현재 좌표 읽기
2. 목표 좌표 계산 (± 1)
3. is_wall(y, x) 호출
4. 벽이면 종료
5. 아니면 좌표 변경

[6-5-1] ghost_up

현재 위치 로드

```
mov al, [ghost_y]  
mov bl, [ghost_x]
```

다음 위치 계산

```
mov dl, al  
dec dl
```

- y-1(위로 이동)

함수 호출 준비

```
movzx rax, dl  
movzx rbx, bl
```

벽 체크

```
call is_wall  
cmp al, 1  
je done
```

- al = 1 이면 벽 → 이동 안함

실제 이동

```
dec byte [ghost_y]
```

- y 값 실제 변경

[6-5-2] ghost_down

```
inc dl
```

- y+1
- 나머지는 동일 구조

[6-5-3] ghost_left/ghost_right 차이

left

```
mov dl, bl
```

```
dec dl
```

- x-1

right

```
inc dl
```

- x+1

[6-6] check_ghost_collision

x 좌표 비교

```
movzx rax, byte [ghost_x]
```

```
movzx rbx, byte [player_x]
```

```
cmp rax, rbx
```

```
jne no_hit
```

y 좌표 비교

```
movzx rax, byte [ghost_y]
```

```
movzx rbx, byte [player_y]
```

```
cmp rax, rbx
```

```
jne no_hit
```

조건

```
ghost_x == player_x
```

```
AND
```

```
ghost_y == player_y
```

충돌 발생

```
mov rax, 1
```

```
ret
```

- true: 충돌

아닐 경우

```
xor rax, rax
```

```
ret
```

– false

[6-7] check_ghost_cross

이 check_ghost_cross 함수가 필요한 이유는 단순히 충돌을 한 번 더 체크하는 것이 아닌, 게임에서 발생하는 시간 단위 이동 문제(프레임 간 충돌 누락)를 해결하기 위해서이다.

일반적인 check_ghost_collision 은 현재 프레임 기준으로 ghost 와 player 가 같은 위치에 있는지만 검사한다.

하지만 게임은 보통 턴 단위나 프레임 단위로 움직이기 때문에, ghost 가 한 칸 이동하는 동안 player 도 동시에 움직이면 두 객체가 서로 위치를 스치고 지나가는 상황이 발생할 수 있다.

예를 들어, ghost 가 (5, 5)에서 (5, 6)으로 이동하고, player 가 동시에 (5, 6)에 있었다고 하자. 단순 collision 만 보면 “ghost 가 (5, 6)에 도착한 뒤 player 와 겹쳤다”는 상황만 잡힌다. 문제는 반대 상황이다. player 가 먼저(5, 6)에 있고 ghost 가 (5, 5)에서 (5, 6)으로 이동하는 “이동하는 중간 과정”에서는 실제로 둘이 충돌했지만, 만약 프레임 처리 순서 때문에 ghost 가 이동 이후에만 검사하면 그 순간 충돌이 누락될 수 있다.

그렇기에 이 함수가 필요한 이유이다.

이 함수는 ghost 의 이전 위치(ghost_prev_x, ghost_prev_y)를 사용해서 “ghost 가 방금 지나간 자리”를 기준으로 player 와 비교한다. 즉, ghost 가 한 프레임 사이에 이동하면서 player 가 그 경로 위에 있었는지를 확인한다.

결국 이 함수는 단순 충돌이 아니라 “이동 중간에 발생하는 충돌(스윙 충돌)”을 잡기 위한 장치이다.

```
check_ghost_cross:
    ; 이전 위치 비교
    movzx rax, byte [ghost_prev_x]
    movzx rbx, byte [player_x]
    cmp rax, rbx
    jne .no

    movzx rax, byte [ghost_prev_y]
    movzx rbx, byte [player_y]
    cmp rax, rbx
    jne .no

    mov rax, 1
    ret

.no:
    xor rax, rax
    ret
```

[6-8] is_wall

- 좌표(y, x)를 맵 인덱스로 변환해서 해당 위치가 벽인지 검사
- #이면 1, 아니면 0 반환

[6-9] get_map_char

- 좌표(y, x)를 맵 인덱스로 변환해서 해당 위치의 문자를 그대로 가져오는 함수
- 벽/길/점 등 맵 상태 확인용

Chapter 4. 결론 및 한계

결론

프로젝트는 x86-64 아키텍처 기반의 저사양 환경에서 NASM 어셈블리 언어를 활용하여 터미널 기반 Text Pacman 게임을 구현함으로써, 저수준 시스템 동작 원리에 대한 실질적인 이해를 목표로 수행되었다.

프로젝트를 통해 단순한 문법 학습을 넘어, 프로그램이 실제 하드웨어와 어떻게 상호작용하는지에 대한 구체적인 흐름을 경험할 수 있었다. 특히 Linux syscall 인터페이스를 직접 활용하여 입출력을 처리하고, 메모리 구조를 기반으로 데이터를 직접 관리하며, 레지스터 단위의 연산을 통해 프로그램의 흐름을 제어하는 과정을 구현함으로써 로우레벨 프로그래밍의 본질적인 구조를 체득할 수 있었다.

또한, 게임 루프 설계, 입력 처리, 충돌 검사, 상태 업데이트와 같은 전반적인 시스템 흐름을 어셈블리 수준에서 직접 구성함으로써, 고수준 언어에서 추상되어 있던 개념들이 실제로 어떻게 구현되는지를 명확하게 이해할 수 있었다. 특히 ANSI Escape Sequence를 활용한 터미널 제어, Raw Mode 기반의 실시간 입력 처리, 그리고 프레임 제한을 통한 실행 흐름 제어는 시스템 프로그래밍 관점에서 중요한 경험으로 작용하였다.

결과적으로 단순한 게임 구현을 넘어, 저수준 환경에서의 설계 방식과 문제 해결 접근법을 체계적으로 이해하는 데 의의가 있으며, 로우레벨 프로그래밍 역량을 실질적으로 향상시키는 계기가 되었다.

한계

프로젝트는 어셈블리 언어 기반으로 게임을 구현하는 데 초점을 맞추었기 때문에, 몇 가지 구조적 및 기능적 한계를 가진다.

1. 터미널 기반 출력 환경의 제약으로 인해 그래픽 표현이 제한적이며, 모든 화면을 문자 단위로 처리하여 확장성에 한계가 존재한다.
2. 현재 구현은 단일 스레드 기반의 동기적 구조로 설계되어 있어, 입력 처리와 게임 로직, 렌더링 순차적으로 수행된다. 이로 인해 높은 반응성과 복잡한 상호작용을 구현하는 데에는 제약이 있다.
3. 일부 알고리즘 구현은 단순화를 위해 제한된 범위에서 동작하도록 설계되었다. 예를 들어, 좌표 처리나 문자열 변환 로직은 특정 범위 내에서만 정상 동작하도록 되어 있다.
4. 코드의 가독성과 유지보수성이 낮으며, 코드 복잡도가 급격히 증가하는 문제가 발생한다.
5. 고스트 이동 로직과 같은 일부 기능은 단순한 랜덤 기반으로 구현되어 있어, 제한적인 동작만을 제공한다.

부록. 기술 설명

[1] 리눅스 터미널 제어와 Raw Mode 구현 분석

tcgetattr(), tcsetattr() 및 어셈블리 수준 제어

[1-1] 서론

리눅스에서 터미널은 단순한 입출력 창이 아니라 커널의 TTY 서브시스템에 의해 관리되는 특수 장치이다.

프로그램은 기본적으로 Canonical Mode(정규 모드)에서 동작하며, 이 모드에서는 다음과 같은 특성이 존재한다.

- Enter 입력 전까지 데이터가 전달되지 않음
- 입력 문자가 자동으로 화면에 출력(ECHO)
- Ctrl + C 입력시 SIGINT 시그널 발생

그러나 텍스트 에디터, 셸, 인터랙티브 프로그램 등은 키 입력을 즉시 처리해야 하므로 Raw Mode 설정이 필요하다.

이를 가능하게 하는 인터페이스가 tcgetattr()와 tcsetattr()이다.

[1-2] 터미널 제어의 기본 구조

터미널 입력 처리 흐름은 다음과 같다.

사용자 키 입력



커널 TTY 드라이버



라인 디서플린(Line Discipline)



프로세스의 stdin (fd = 0)

라인 디서플린은 입력 처리 방식을 제어하며,
이 설정은 struct termios 구조체를 통해 관리된다.

[1-3] 파일 디스크립터(fd)의 의미

리눅스에서는 모든 입출력이 파일 디스크립터로 관리된다.

fd 값	의미
0	표준 입력 (stdin)
1	표준 출력 (stdout)
2	표준 에러 (stderr)

터미널 제어는 일반적으로 stdin(fd=0)을 대상으로 수행한다.

[1-4] tcgetattr 함수 분석

[1-4-1] 함수 원형

```
int tcgetattr(int fd, struct termios *termios_p);
```

[1-4-2] 기능

- 현재 터미널 설정을 읽어
- termios_p 구조체에 복사한다.

[1-4-3] 사용 목적

Raw Mode 설정을 위해서는 기존 설정을 백업한 뒤 수정해야 한다.

```
struct termios old_termios;  
tcgetattr(0, &old_termios);
```

[1-5] tcsetattr 함수 분석

[1-5-1] 함수 원형

```
int tcsetattr(int fd, int optional_actions, const struct termios *termios_p);
```

[1-5-2] optional_actions 값

값	의미
TCSANOW	즉시 적용
TCSADRAIN	출력 완료 후 적용
TCSAFLUSH	출력 완료 후 입력 버퍼 제거 후 적용

Raw Mode 설정에서는 일반적으로 TCSANOW 를 사용한다.

[1-6] termios 구조체 분석

```
struct termios {  
    tcflag_t c_iflag;  
    tcflag_t c_oflag;  
    tcflag_t c_cflag;  
    tcflag_t c_lflag;  
    cc_t c_line;  
    cc_t c_cc[NCCS];  
};
```

각 필드는 비트 플래그 집합이다.

x86-64 glibc 기준 메모리 레이아웃:

필드	offset
c_iflag	0
c_oflag	4
c_cflag	8
c_lflag	12

따라서 어셈블리에서

```
mov eax, [new_termios + 12]
```

는 c_lflag 를 읽는 명령이다.

터미널 구조체 확인 방법

```
cat /usr/include/x86_64-linux-gnu/bits/termios-struct.h
```

[1-7] ICANON, ECHO 플래그 값의 근원

헤더 파일 <termios.h> 내부 정의:

```
#define ICANON 0000002
#define ECHO 0000010
```

이 값들은 8 진수(octal)로 정의되어 있다.

16 진수로 변환하면:

이름	16진수
ICANON	0x0002
ECHO	0x0008

OR 연산 결과

```
0x0002 | 0x0008 = 0x000A
```

따라서:

```
and eax, ~0x000A
```

는 ICANON 과 ECHO 비트를 동시에 제거한다.

이는 C 코드로 표현하면:

```
new_termios.c_lflag &= ~(ICANON | ECHO);
```

실제 값 확인 방법

```
grep ICANON /usr/include -r
grep ECHO /usr/include -r
```


[1-8] Raw Mode 설정 과정

[1-8-1] 기존 설정 백업

```
struct termios old_termios;  
tcgetattr(0, &old_termios);
```

[1-8-2] 구조체 복사

어셈블리 예시:

```
mov rcx, 60  
mov rsi, old_termios  
mov rdi, new_termios  
  
.copy:  
    mov al, [rsi]  
    mov [rdi], al  
    inc rsi  
    inc rdi  
    loop .copy
```

이는 60 바이트 메모리 복사이며, C 코드로는 다음과 같다.

```
memcpy(&new_termios, &old_termios, sizeof(struct termios));
```

[1-8-3] 플래그 제거

```
mov eax, [new_termios + 12]  
and eax, ~0x000A  
mov [new_termios + 12], eax
```

[1-8-4] 적용

```
tcsetattr(0, TCSANOW, &new_termios);
```

[1-9] 어셈블리 호출 규약 분석

x86-64 System V ABI:

인자	레지스터
1번째	rdi
2번째	rsi
3번째	rdx

따라서:

```
mov rdi, 0
mov rsi, termios_addr
call tcgetattr
```

는

```
tcgetattr(0, &termios);
```

과 동일하다.

rdi=0 은 stdin 을 의미한다.

[1-10] loop 명령어 동작 원리

```
loop .copy
```

의 내부 동작:

```
rcx = rcx - 1
if (rcx != 0)
    jump .copy
```

즉, rcx 는 자동으로 감소한다.

[1-11] 결론

1. tcgetattr()는 현재 터미널 설정을 읽는다.
2. tcsetattr()는 수정된 설정을 적용한다.
3. Raw Mode 는 ICANON, ECHO 등의 비트를 제거하여 구현된다.
4. 플래그 값은 커널 UAPI 헤더에 정의되어 있다.
5. 구조체 offset 은 ABI 에 의해 결정된다.
6. 어셈블리에서는 rdi 를 통해 fd 를 전달한다.
7. loop 명령은 rcx 를 자동으로 감소시키는 반복 명령이다.

Raw Mode 설정은 단순한 사용자 공간 조작이 아니라,
커널의 터미널 처리 방식을 직접 변경하는 작업이다.

[2] ANSI Escape Sequence 기반 커서 이동 루틴의 구조 및 자릿수 처리 원리 분석

[2-1] 작성 목적

x86-64 Linux 환경에서 작성된 `move_cursor` 어셈블리 루틴의 동작 원리와 구현 방식을 상세히 설명하기 위해 작성되었다.

특히 다음 사항을 중점적으로 설명한다.

- ANSI Escape Sequence 기반 커서 이동 원리
- 숫자를 ASCII 문자열로 변환하는 방식
- 자릿수(1 자리, 2 자리, 3 자리 이상)에 따라 구현 방식이 달라지는 이유
- 현재 구현의 구조적 한계

해당 루틴은 터미널 상에서 커서를 특정 좌표(y, x)로 이동시키기 위해 ANSI Escape Sequence 를 직접 생성하고 `write` 시스템콜을 통해 출력하는 기능을 수행한다.

[2-2] 이론적 배경

[2-2-1] 터미널 커서 이동 원리

리눅스 터미널은 ANSI Escape Sequence 를 통해 화면 제어를 수행한다.

커서 이동 명령의 형식은 다음과 같다.

ESC [<row> ; <col> H

ASCII 값으로 표현하면

구성 요소	의미	ASCII 값
ESC	Escape 문자	27
[	제어 시작	91
row	행 번호	ASCII 숫자
;	구분자	59
col	열 번호	ASCII 숫자
H	커서 이동 명령	72

예시:

ESC[5;10H

5 행 10 열로 커서 이동

[2-3] 함수 개요

함수명

move_cursor

레지스터	의미
rdi	y 좌표 (row)
rsi	x 좌표 (column)

출력

- ANSI 제어 문자열 생성
- write 시스템 콜을 통해 STDOUT 출력
- 커서 이동 수행

[2-4] 숫자 → 문자열 변환이 필요한 이유

터미널은 정수 값 자체를 이해하지 못한다.

예:

25

는 CPU 내부에서 이진수로 저장된다.

하지만 터미널은 다음 형태를 요구한다.

‘2’ ‘5’

즉, 정수를 ASCII 문자로 변환해야 한다.

이 과정이 이 루틴의 핵심이다.

[2-5] 자릿수에 따라 구현 방식이 달라지는 이유

[2-5-1] 숫자는 “가변 길이 데이터”이기 때문

정수는 자릿수에 따라 필요한 문자 수가 달라진다.

값	자릿수	필요한 문자 수
5	1자리	1
25	2자리	2
125	3자리	3

하지만 메모리는 고정 크기 공간이다.

따라서 다음 문제가 발생한다.

- 몇 개의 문자를 써야 하는지 런타임에 결정해야 한다.

[2-5-2] 1 자리수(0~9)

예: 7

십의 자리 = 0

일의 자리 = 7

이 경우

“7”

만 출력해야 한다.

만약

“07”

을 출력하면 의미는 같지만 불필요한 0 이 추가된다.

따라서 코드에서는

```
cmp rax, 0
```

```
je .skip_tens
```

를 통해 십의 자리가 0 이면 출력하지 않는다.

[2-5-3] 2 자리 수(10~99)

예: 25

$25 \div 10$

몫 = 2

나머지 = 5

십의 자리와 일의 자리를 각각 ASCII 로 변환하여 출력한다.

이 경우:

“25”

총 2 바이트가 추가된다.

[2-5-4] 3 자리 수 이상(100 이상)

예: 125

현재 구현 방식은:

$125 \div 10$

몫 = 12

나머지 = 5

여기서 문제는 몫이 12 가 되어버린다.

즉, 십의 자리 개념이 깨진다. 왜냐하면 10 으로 한 번만 나누는 방식은 2 자리까지만 정확하기 때문이다.

[2-6] 현재 구현이 2 자리까지만 지원하는 이유

현재 코드의 핵심 변환 로직:

```
div 10
```

이 연산은

```
rax = 몫  
rdx = 나머지
```

을 생성한다.

이 방식은

- rdx: 항상 0~9(일의 자리)
- rax: 0~9(십의 자리) → 2 자리까지만 유효

하지만 입력이 100 이상이면:

- rax 가 10 이상이 될 수 있음
- 즉, rax 자체도 다시 분해해야 함

현재 코드는 이를 처리하지 않기 때문에 3 자리 이상에서 오동작한다.

[2-7] 일반적인 자릿수 처리 방식

3 자리 이상을 처리하려면 다음 구조가 필요하다.

반복 나눗셈 구조

```
while(value > 0)  
    digit = value % 10  
    value = value / 10
```

이렇게 하면

- 125 → 5 → 2 → 1
- 역순으로 저장 필요

즉, 자릿수가 늘어날수록 반복 구조가 필요하다.

현재 구현은 고정 2 자리 전용 최적화 버전이다.

[2-8] move_cursor 함수

이 함수는 (rdi = y, rsi = x)를 입력 받아

```
ESC[y;xH
```

형태의 ANSI Escape 문자열을 직접 만들어 write 시스템콜로 터미널에 출력한다.
즉, 터미널 커서를(y,x) 위치로 이동시키는 함수이다.

[2-8-1] ESC 와 '[' 저장

```
mov byte [cursor_buf], 27  
mov byte [cursor_buf+1], '['
```

- 27 = ASCII ESC

- '[' = ANSI 제어 시퀀스 시작 문자

이제 버퍼 상태는:

```
[0] = ESC
```

```
[1] = '['
```

현재 문자열:

```
ESC[
```

[2-8-2] Y 좌표를 10 으로 나누기

```
mov rax, rdi    ; y  
mov rbx, 10  
xor rdx, rdx  
div rbx
```

div rbx 는 다음을 수행한다.

```
RDX:RAX ÷ RBX
```

우리는 RDX 를 0 으로 초기화했으므로:

```
y ÷ 10
```

레지스터	의미
rax	몫 → 십의 자리
rdx	나머지 → 일의 자리

예: y = 25

```
rax = 2
```

```
rdx = 5
```

[2-8-3] 버퍼 인덱스 초기화

```
mov rcx, 2
```

이미

```
cursor_buf[0]  
cursor_buf[1]
```

에 ESC, '['를 썼으므로 다음에 쓸 위치는 index 2
rcx 는 “현재까지 작성된 길이”이자 “다음 쓸 위치”이다.

[2-8-4] Y 의 십의 자리 출력

```
cmp rax, 0  
je .skip_y_tens
```

십의 자리가 0 이면 출력하지 않는다.

예: $y = 5$

```
십의 자리 = 0  
일의 자리 = 5
```

“05” 대신 “5”만 출력하기 위함

출력하는 경우:

```
add al, '0'  
mov [cursor_buf+rcx], al  
inc rcx
```

0 을 더하는 이유는

ASCII 에서

```
'0' = 48
```

숫자 $2 + 48 =$ 문자 '2'

즉, 숫자를 문자로 변환하는 과정이다.

[2-8-5] Y 의 일의 자리 출력

```
add dl, '0'  
mov [cursor_buf+rcx], dl  
inc rcx
```

일의 자리는 항상 출력한다.

현재 문자열 예시($y = 25$)

```
ESC[25
```


[2-8-6] ‘;’ 추가

```
mov byte [cursor_buf+rcx], ';'
inc rcx
```

현재

```
ESC[25;
```

[2-8-7] x 좌표 처리

```
mov rax, rsi
mov rbx, 10
xor rdx, rdx
div rbx
```

결과:

```
rax = 십의 자리
rdx = 일의 자리
```

[2-8-8] 십의 자리 조건부 출력

```
cmp rax, 0
je .skip_x_tens
```

0 이면 생략

[2-8-9] 일의 자리 출력

```
add dl, '0'
mov [cursor_buf+rcx], dl
inc rcx
```

현재 예시:

```
ESC[25;10
```

[2-8-10] ‘H’ 추가

```
mov byte [cursor_buf+rcx], 'H'
inc rcx
```

최종 완성

```
ESC[25;10H
```

이 문자열이 터미널에 전달되면 커서가 해당 위치로 이동한다.

[2-9-11] 시스템 콜 부분

```
mov rax, SYS_write
mov rdi, STDOUT
lea rsi, [cursor_buf]
mov rdx, rcx
syscall
```

여기서 핵심은

```
mov rdx, rcx
```

rcx 는 우리가 만든 문자열 길이이다. 즉 정확한 길이만 출력한다.
하드코딩된 6 이나 8 을 쓰지 않았다.

[3] 정수 → 10 진수 문자열 변환

[3-1] 개요

정수 값을 10 진수 ASCII 문자열로 변환하는 알고리즘에 대해 설명한다.

특히, 버퍼의 끝에서부터 문자열을 생성하는 방식(Reverse Fill Strategy)의 원리와 설계 이유를 다룬다.

[3-2] 문제 정의

정수 값 1234 를 다음과 같은 문자열로 변환한다.

“1234”

- printf 사용 불가
- 동적 메모리 사용 없음
- 고정 버퍼 사용
- syscall 기반 출력 환경

[3-3] 기본 원리

10 진수는 자리수 체계를 기반으로 한다.

정수 n 에 대해 다음이 성립한다.

마지막 자리 = $n \% 10$

나머지 값 = $n / 10$

이 연산을 반복하면 각 자릿수를 분해할 수 있다.

예:

$1234 \% 10 = 4$

$123 \% 10 = 3$

$12 \% 10 = 2$

$1 \% 10 = 1$

결과 생성 순서

$4 \rightarrow 3 \rightarrow 2 \rightarrow 1$

즉, 최하위 자리부터 생성된다.

[3-4] 설계 문제

우리가 원하는 출력 문자열은

“1234”

하지만 알고리즘이 생성하는 방식은 순서가 반대다.

[3-5] 해결 방법 - Reverse Fill

버퍼의 끝에서부터 데이터를 채운다.

동작 방식

1. 버퍼 끝을 가리키는 포인터 설정
2. $n \% 10$ 결과를 ASCII 로 변환
3. 포인터를 감소시키며 저장
4. $n == 0$ 이 될 때까지 반복

[3-6] 메모리 구조

버퍼 정의

score_buf: 32 bytes

메모리 구조:

[0] ... [26] [27] [28] [29] [30] [31]

처리 과정(score=1234)

초기:

rdi $\rightarrow$ score_buf + 30

변환 후:

[26]='1'
[27]='2'
[28]='3'
[29]='4'
[30]=0 (NULL)

출력 시작 주소:

rdi = 26

출력 길이

$(30 - 26) = 4\text{bytes}$

[3-7] 코드

```
section .bss
score_buf resb 32

section .text
draw_score:

mov rax, [score]
mov rbx, 10
lea rdi, [score_buf + 30]
mov byte [rdi], 0
dec rdi

.convert:
xor rdx, rdx
div rbx
add dl, '0'
mov [rdi], dl
dec rdi
test rax, rax ; rax & rax = rax 가 0 인지 아닌지 검사, 결과가 0 이면 ZF = 1
jnz .convert ; Jump if Not Zero, rax 가 0 이 아니면 점프

inc rdi

; 출력
mov rax, SYS_write
mov rsi, rdi
mov rdx, score_buf + 30
sub rdx, rdi
mov rdi, STDOUT
syscall

ret
```

[4] 프레임 제한과 나노초 단위 시간

[4-1] 시간 단위와 나노초(Nanoseconds, ns)

컴퓨터 내부에서 시간은 극히 작은 단위까지 측정할 수 있다. 일반적인 시간 단위를 나열하면:

단위	약어	1단위
초	s	기본 시간 단위
밀리초	ms	1ms = 0.001s = 1,000,000ns
마이크로초	μs	1μs = 0.000001s = 1,000ns
나노초	ns	1ns = 0.000000001s = 1초의 10억분의 1

[4-2] 컴퓨터 속도와 나노초

현대 컴퓨터의 CPU 클럭 속도를 나노초와 연결하여 본다.

CPU 클럭	클럭 주기	예시
3GHz	1 / 3,000,000,000 = 0.33ns	1초에 30억번 신호 처리
1GHz	1 / 1,000,000,000 = 1ns	1초에 10억번 신호 처리

즉, 나노초 단위는 CPU가 한 사이클 동안 작업할 수 있는 시간과 비슷하다.

- 100ms = 100,000,000ns → 3GHz CPU 기준 3억 클럭 사이클 동안 대기 가능

[4-3] Linux에서 프레임 제한 구현: nanosleep

게임이나 그래픽 루프에서 프레임 제한을 위해 nanosleep 시스템 콜을 사용할 수 있다.

```
struct timespec {  
    time_t tv_sec;    // 초 단위  
    long   tv_nsec;   // 나노초 단위 (0 ~ 999,999,999)  
};
```

- tv_sec: 정수 초

- tv_nsec: 추가 나노초

예: 100ms 대기 → tv_sec = 0, tv_nsec = 100,000,000 = 0.1 초, 1억

x86-64 Linux ASM 예제

```
section .data
timespec:
    dq 0
    dq 1000000000
section .text
global frame_limit
fram_limit:
    mov rax, 35      ; SYS_nanosleep
    lea rdi, [timespec] ; timespec 구조체 주소
    xor rsi, rsi      ; rem = NULL(남은 시간 저장 안함)
    syscall
    ret
```

- rax = 35 → nanosleep syscall 번호
- rdi → 요청 시간(timespec 구조체)
- rsi → 남은 시간(rem 구조체, 여기서는 사용하지 않음)
- 호출 후 100ms 동안 CPU를 블록하여 루프 속도 제어 가능

[4-4] 프레임 제한 개념

게임이나 시뮬레이션 루프에서

1. 프레임 시작
2. 로직 처리
3. 렌더링
4. 남은 시간 동안 대기 (nanosleep)

목표: 매 프레임 일정 시간 유지 → 목표 FPS 달성

- 100ms 프레임 제한 → 최대 10FPS
- 16.67ms 프레임 제한 → 최대 60FPS

[4-5] FPS

[1] (Frame Per Second)란

- FPS는 초당 프레임 수를 의미한다.
- 쉽게 말해, 1초 동안 화면이 몇 번 갱신되는지 표시한 값이다.

$$FPS = \frac{1}{1 \text{ 프레임 시간(초)}}$$

예시:

1프레임 시간	FPS
16.67ms	60FPS (게임 대부분 목표)
100ms	10FPS
33.33ms	30FPS

즉, 100ms 프레임 제한을 걸면 1초에 10번만 화면이 갱신된다.

[2] 100ms 프레임 제한과 나노초

- 100ms = 0.1 초 = 100,000,000ns
- nanosleep 로 100ms 대기하면 루프를 0.1 초 단위로 제한하는 것과 동일

$$FPS = \frac{1}{0.1} = 10$$

- 즉, 최대 FPS = 10 → 1 초에 10 프레임만 처리 가능

[3] 100ms 가 CPU GHz 와 어떤 관계

- 예: CPU 클럭 3GHz → 1 초에 30 억 사이클 수행 가능

$$1 \text{ 클럭 주기} = \frac{1}{3GHz} = 0.33ns$$

- 100ms = 100,000,000ns
- 3GHz CPU 기준 몇 사이클 동안 대기하는지 계산:

$$\text{사이클 수} = \frac{100,000,000ns}{\frac{0.33ns}{\text{클럭}}} \approx 303,030,303 \text{ 클럭}$$

즉, 100ms 동안 CPU 는 약 3 억 클럭 주기 동안 아무것도 하지 않고 대기하는 셈이다.
(1 초에 30 억번)

- 그래서 프레임 제한은 CPU 사용을 절약하고, 화면 갱신 속도를 일정하게 유지하는 효과가 있다.

[4] FPS x CPU 클럭 쉽게 연결하기

- 예: 100ms 프레임 제한 → 10FPS
- 3GHz CPU → 1 초에 30 억 클럭
- 1 프레임 동안 CPU 가 쉴 시간 = 0.1 초 → 약 3 억 클럭 사용 안함